

Software Dependability

Intro to Software Testing and Test Case Selection

The Fundamental Principle of Dependability

“If all faults to which a system is subject are identified and their effects are properly mitigated, the system will meet its dependability requirements.”

There are different ways to mitigate faults; they can be

- **Avoided:** If we build systems without faults, we do not need to do anything else.
- **Eliminated:** Do you remember software testing?
- **Tolerated:** Managed in a way that they do not cause failures.
- **Forecast:** Suppose when their effects will occur and be prepared for them.

Since not all projects are the same and perfection is not possible, we need to address dependability issues.

This week, we start digging into fault elimination.

Software Testing: Motivation and Definition

Developers eventually introduce defects that can cause failures, producing loss of money or even being mortal.

Some defects are easy to spot, but in most cases, they hide.

There are two initial options to discover defects: (1) **code review** done by developers, which is a preventive action aimed at finding defects before putting the code into production; and (2) **waiting until the defect is exploited**, causing a failure and, at that moment, **debug** the code.

However, we aim to find defects **as soon as possible**, not when externals use the code: **debugging is only an a-posteriori activity** to be made once the defect is found. **Code review is an a-priori action**, yet humans do it, sometimes in a limited time, and with no automated mechanisms to continuously check source code quality.

Testing can be defined as “the set of activities and disciplines that can support developers when checking that the expected outcome of a piece of software is produced”.

Testing is dynamic, i.e., it involves the execution of a software component to evaluate one or more of its properties.

Software Testing: Different Perspectives, Different Objectives

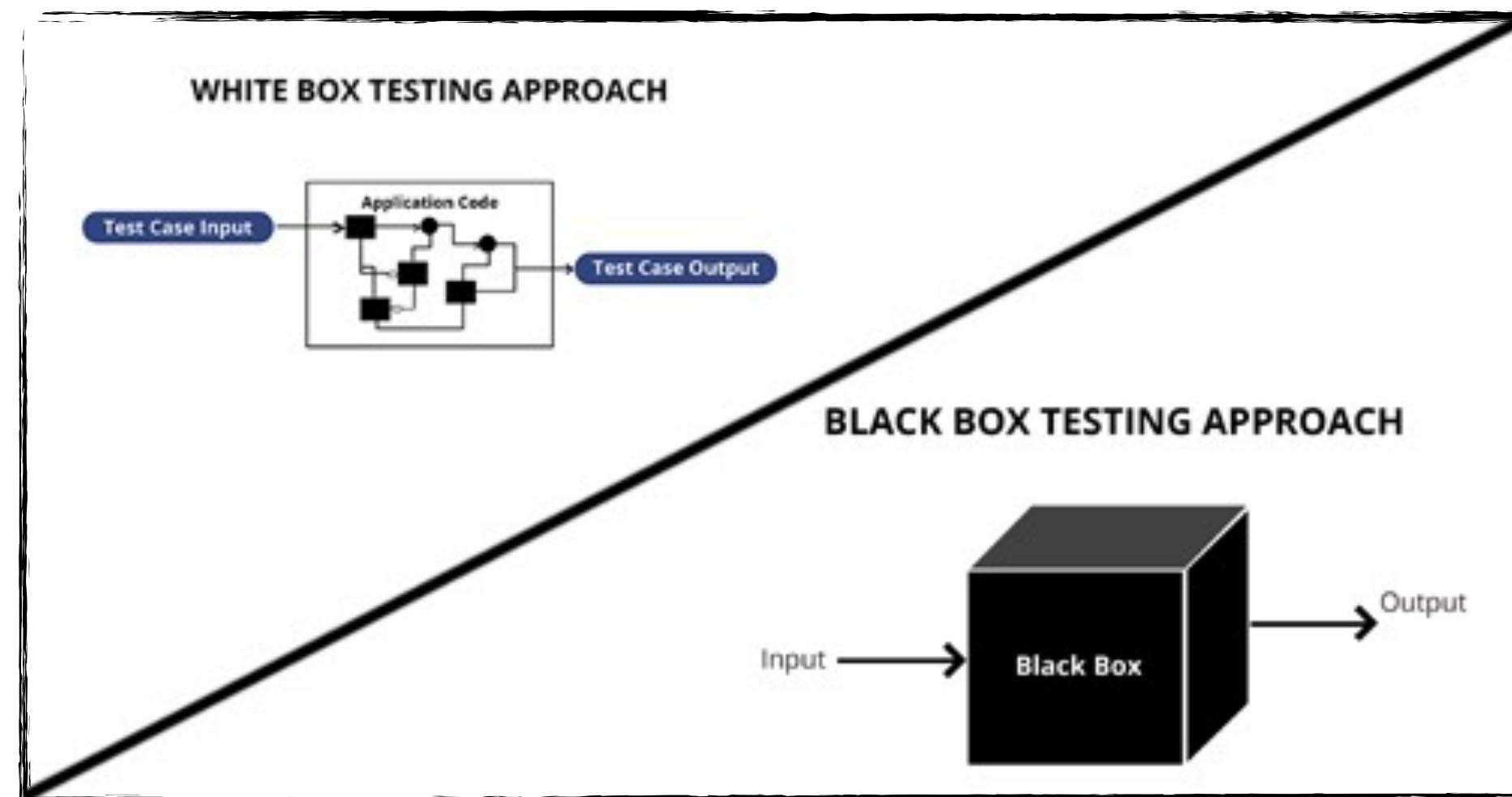
Developers. The only goal of developers is to exploit software testing to identify defects in the code being developed.

Quality Assurance Engineers. Software testing evaluates the overall quality of the software being produced.

Testers. Software testing is used to check the implemented functionalities against the corresponding requirements.

Customers. In this case, they aim to verify the usability and applicability of the functionalities implemented in the code.

Software Testing: Methods

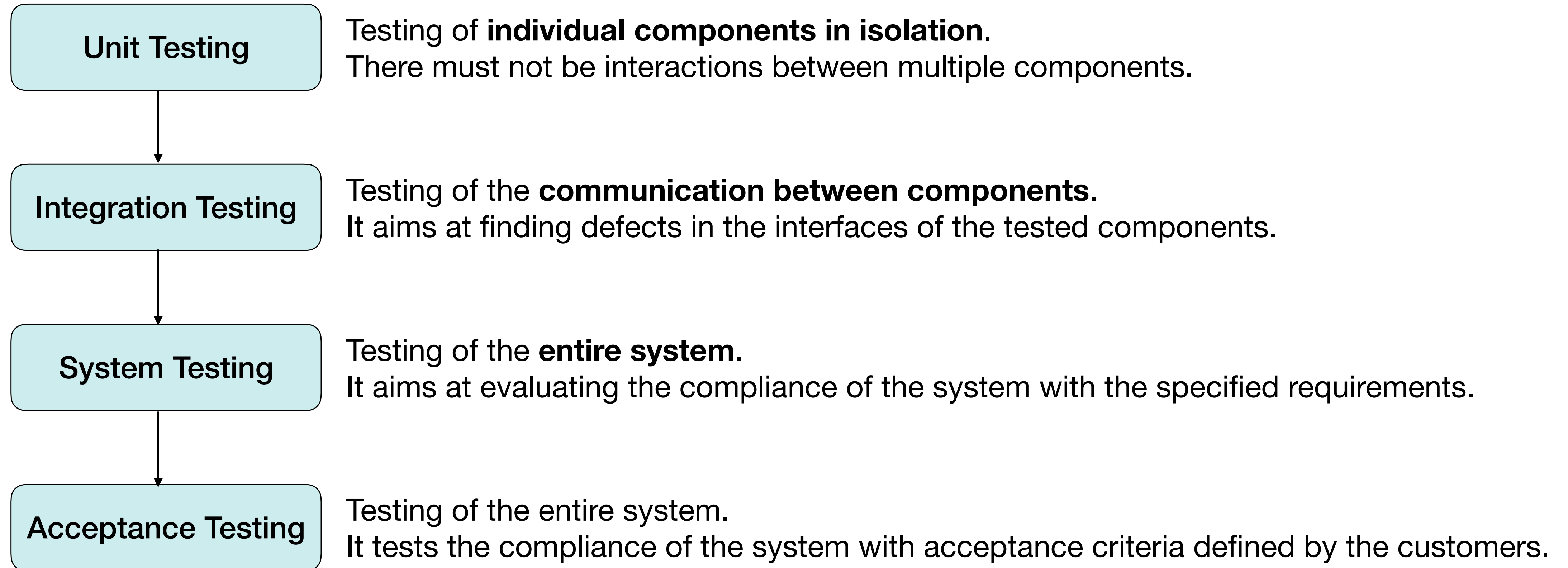


Black-box. Also called specification-based testing, it aims at testing functionalities by observing their external behavior.

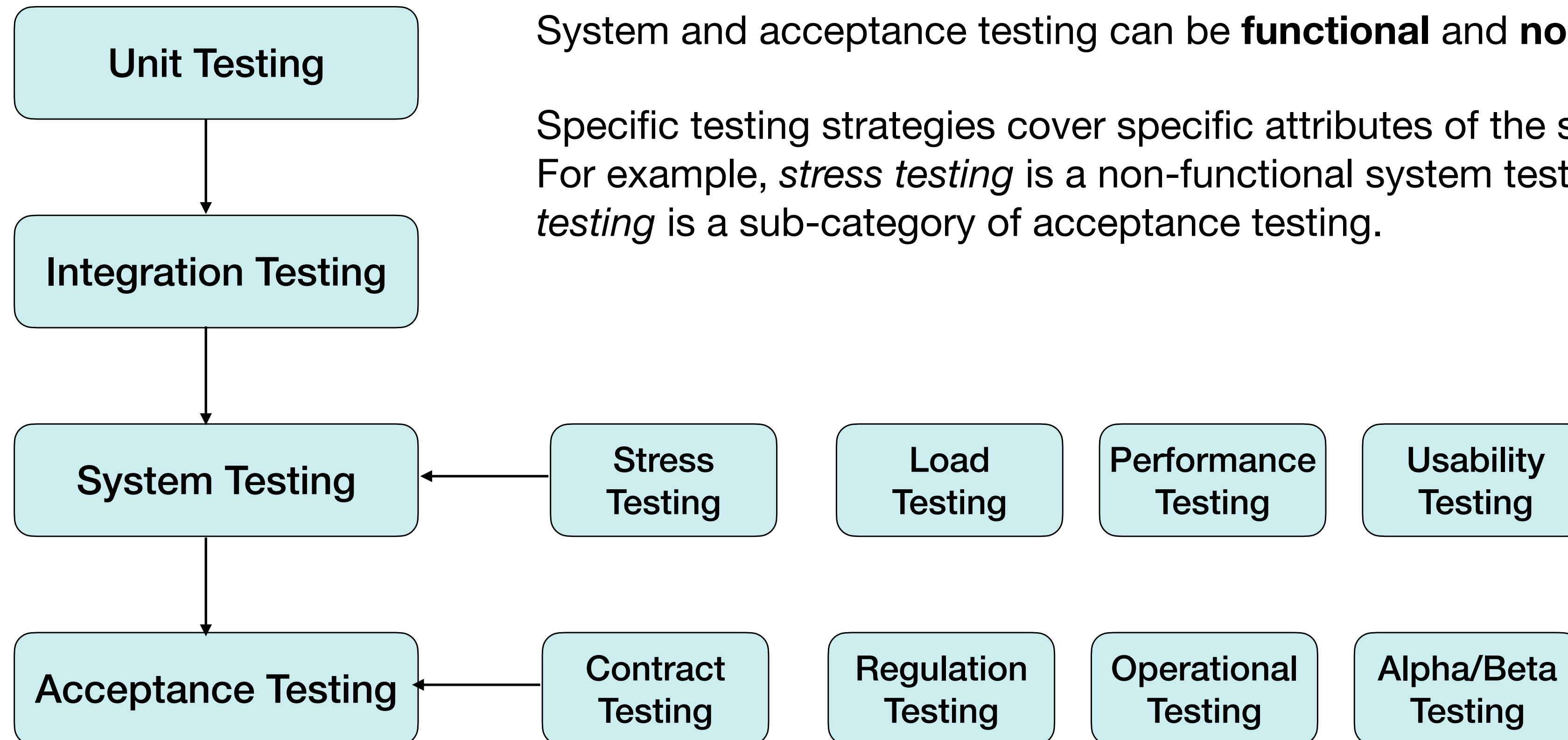
White-box. Also called structural-based testing, it aims at examining the internal structure of the code to derive tests.

Key difference: How developers have to design test cases.

Software Testing: Test Levels



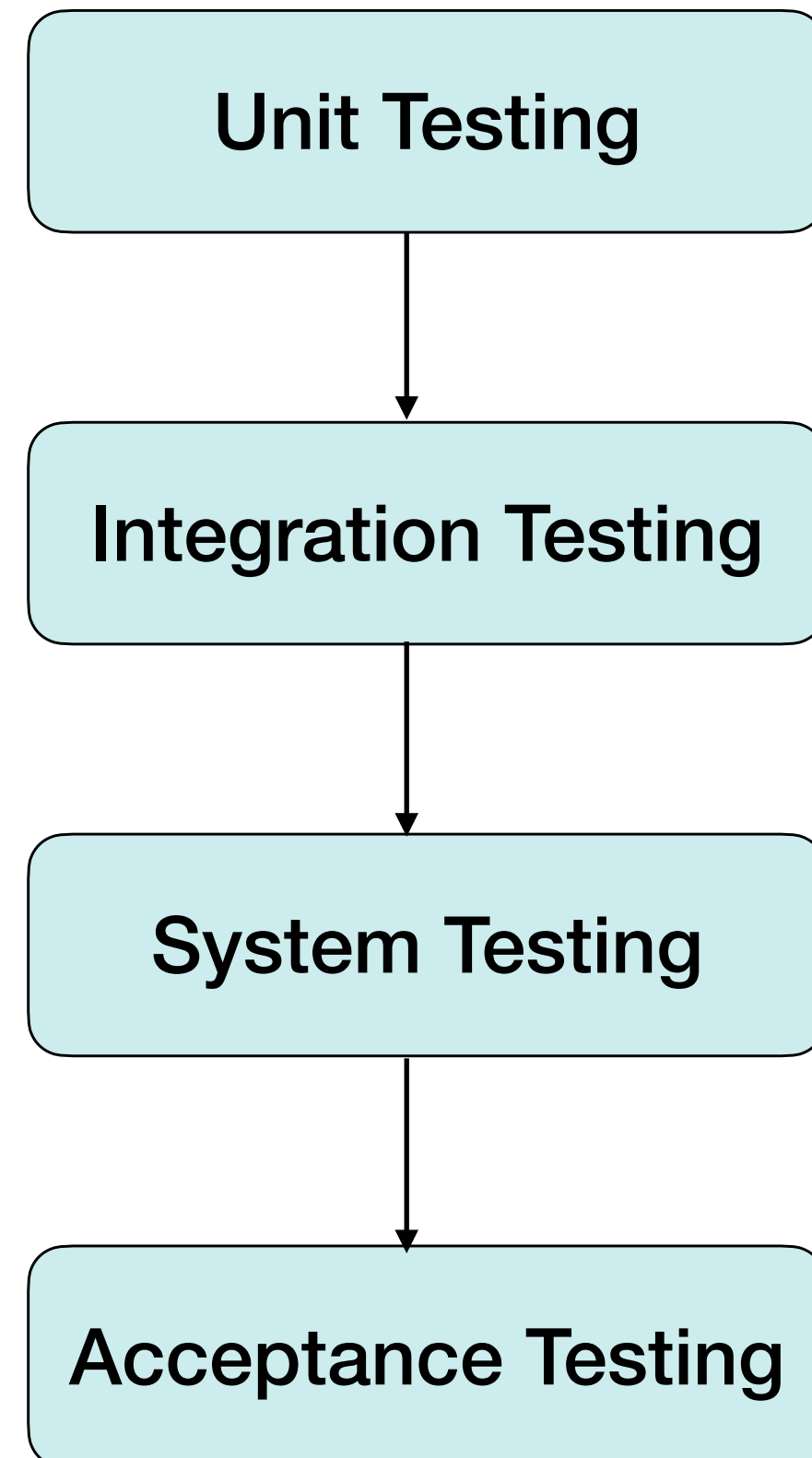
Software Testing: Test Levels



System and acceptance testing can be **functional** and **non-functional**.

Specific testing strategies cover specific attributes of the software being developed. For example, *stress testing* is a non-functional system testing type, while *alpha/beta testing* is a sub-category of acceptance testing.

Software Testing: Test Levels



Q. Shall we run all test cases **all the time**?

A. No, this is **unfeasible**. Modern systems have thousands of tests that require time. **Regression testing**. It runs a subset of tests when a new change is developed and committed to the central repository. The subset of tests is selected based on the modified code, i.e., all tests connected to the change are run.

Q. Shall we continuously run **non-functional** tests?

A. Generally, not. It may depend on the specific system being developed, yet in most cases, non-functional tests can be **periodically** run, e.g., close to a new release.

Software Testing: The Need for Planning

A **test plan** is a systematic approach to test a software system, containing a detailed analysis of **the testing workflow**.

The **Master Test Plan** document (MTP) presents the overall approach to all testing levels. For cyber-physical systems, it should cover the entire product, including hardware components. The document can include other evaluation techniques, e.g., code review, walkthroughs, and inspections.

The MTP is usually accompanied by **more specific test plans**, one for each testing level under consideration.

One of the key sections of the MTP concerns the **features to be tested**. In a large-scale project, the number of features could be too many to be systematically exercised, so there is a need for some selection strategies. The plan must consider (i) **the risks associated with the features of the system** and (ii) **the resources available for testing**.

The MTP also documents **the features not to be tested and their associated risk**.

It maintains the **rationale**, and further modifications are allowed in case newer resources become available.



Software Testing: The Need for Planning

The core component of the MTP is the **specification of the testing approach**, which is defined by multiple factors, e.g.,

Test requirements

These are established based on the application requirements, e.g., data-intensive systems have testing requirements and objectives different from mobile applications.

Test objectives

Test environment

Test designers must specify the environmental conditions required to achieve the objectives. They can both influence and be influenced by the available resources that can be acquired.

Resources

Testing tools

Based on the environment and resources available, test designers define which tools can be put in place, which could be acquired, and how production code will be subject to code review.

Review strategy

In this stage, requirements and resources available can depend on and might influence the **selection of the components of the shelf** (COTS) to be used, e.g., third-party libraries which are already tested.

Depending on the requirements, **non-functional testing can be planned and scheduled.**

Software Testing: Criteria

Once describing the testing approach, testers must specify the **pass/fail criteria** to test each feature.

Typically, pass/fail criteria are expressed in terms of (i) test cases passed and failed, (ii) number, type, severity, and location of defects, (iii) usability, reliability, or stability of specific non-functional aspects.

The specification of pass/fail criteria is not trivial because **it is difficult when to stop testing**.

Specific pass/fail criteria vary from organization to organization.

Suspension criteria are conditions that warrant a temporary suspension of testing and the criteria for resumption.

Incomplete tasks on the critical path, large volumes of defects, the identification of critical defects, incomplete testing environments, or resource shortages are typical suspension criteria.

Some testing tasks have dependencies. Testing a component could be useless if it depends on another.

Acceptance Criteria are widely accepted in modern software systems, including **the conditions for which specific test cases or functionalities can be included in the codebase**.

Many software systems explicitly refer to **testing-related metrics that must be considered when merging pull requests** (e.g., how high code coverage should be to consider a new functionality sufficiently tested?). Usually, this information is available in the contribution guidelines.

Generally, acceptance criteria refer to all metrics and measures expressing the satisfactory testing of either **single functionalities, the entire software system, or specific testing activities**.

Software Testing: Testing Tasks and Responsibilities

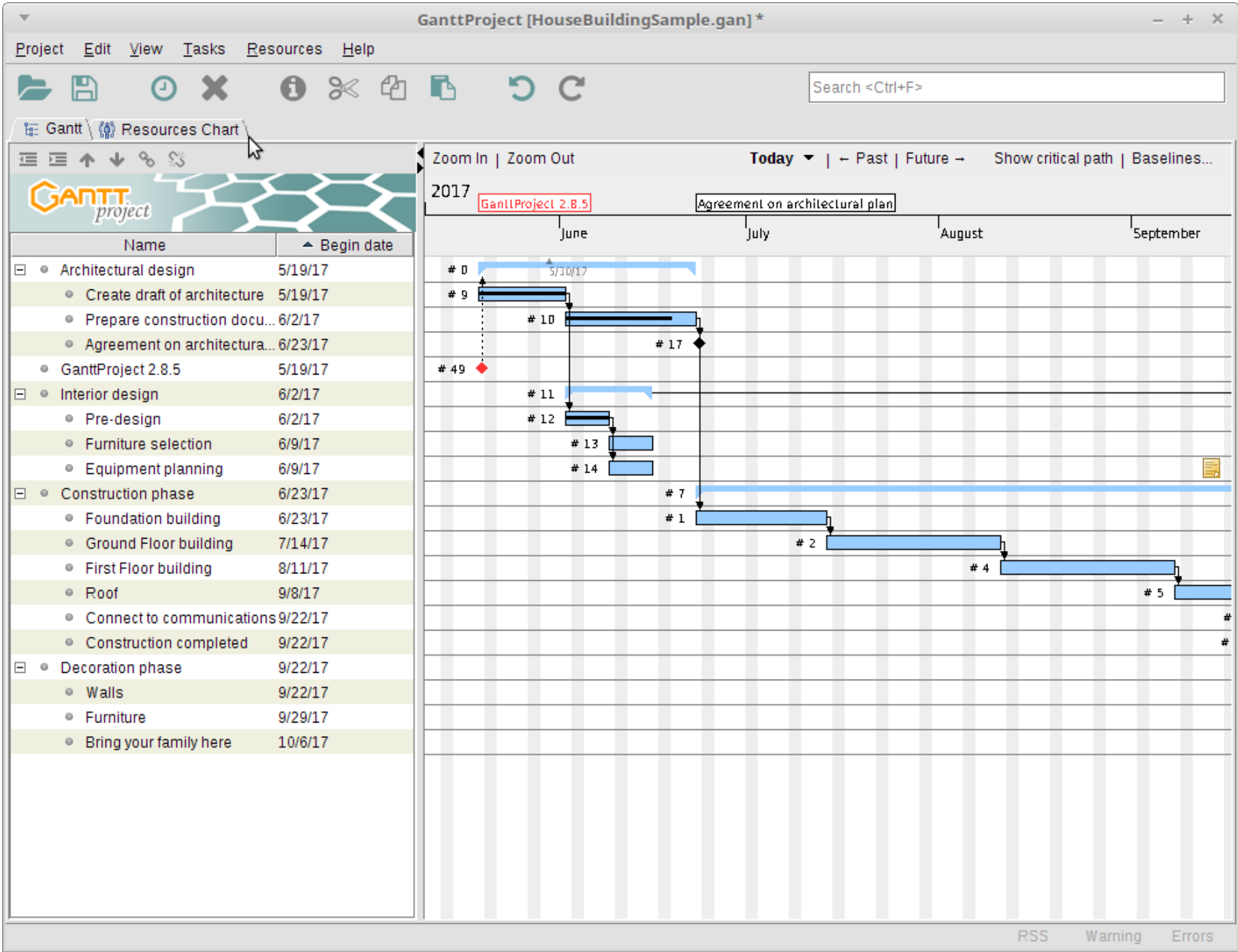
The MTP should contain a **responsibility matrix** where tasks are put on rows and testing staff on columns.

The matrix describes **task responsibilities** and helps understand **whether additional staff members are needed**.

Finally, a **Gantt chart** is included to **schedule** the testing activities and control the process workflow.

Risks are also considered and analyzed.

RESPONSIBILITY ASSIGNMENT MATRIX								
	PERSON 1	PERSON 2	PERSON 3	PERSON 4	PERSON 5	PERSON 6	PERSON 7	PERSON 8
Work package 1	R		C		A	I		
Work package 2		A	I				A	R
Work package 3		C		R		A	I	
Work package 4	I		A		C	R		

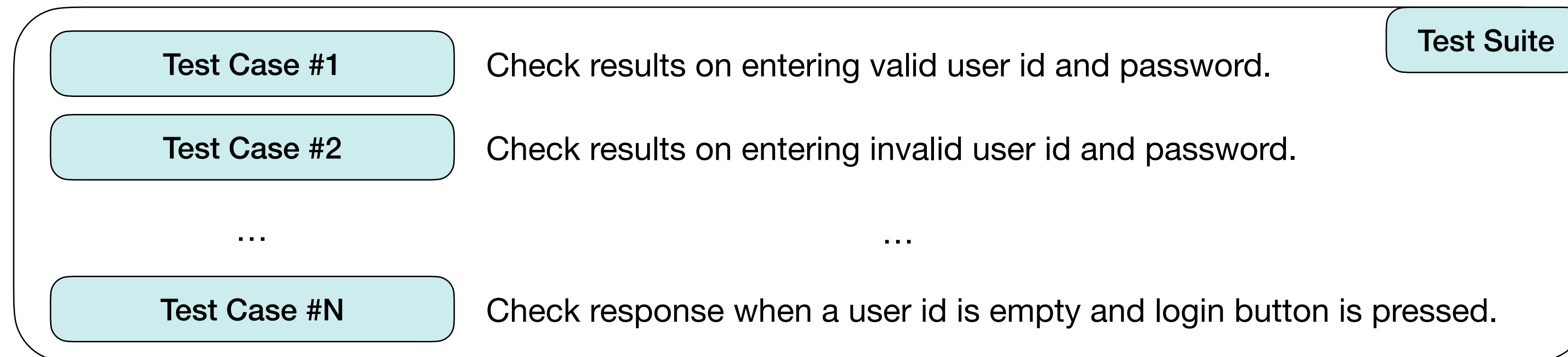


Software Testing: Writing Test Cases

According to the IEEE Standard 610, a **test case** is represented by “a set of inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or to verify the compliance with a specific requirement”.

A **test suite** is a collection of test cases aiming to exercise specific objects of the production code.

Example: Testing a Login function.



A test case not only exercises a functionality but may be built to test **non-functional aspects of source code**.

Software Testing: Designing Test Cases

When designing tests, developers have to implement the testing approach described in the Master Test Plan.

Test cases can be designed using a **black- or white-box philosophy**.

Black-box Testing

No knowledge of the implementation is required.

Test cases are developed based on the expected input of a functionality.

It can identify missing functionalities.

Some techniques.

- *Equivalence Class Testing*: Test cases should cover data of each partition at least once.
- *Boundary Value Testing*: Each input is tested at both ends of its valid range(s) and just outside its valid range(s).

White-box Testing

Knowledge of the implementation is required.

Test cases are developed based on the structure of implemented source code

It can identify edge cases.

Some techniques.

- *Statement Testing*: Test cases should cover all lines of production code.
- *Branch Testing*: Test cases should cover all branches of production code.
- *Condition Testing*: Test cases should cover all conditions of production code.

Software Testing: Designing Test Cases

When designing tests, developers have to implement the testing approach described in the Master Test Plan.

Test cases can be designed using a **black- or white-box philosophy**.

Black-box Testing

Example: Testing a numeric input.

The input can take three “classes” of inputs.

It can identify missing functionalities.

- i) Numbers greater than 0;
- ii) Numbers lower than 0;
- iii) The number 0.

An equivalence class black box strategy consists of writing three test cases, namely those testing the following inputs:

- i) 5; ii) -4; iii) 0.

White-box Testing

Example: Testing the following method.

```
if (x == 0) {  
    x=3;  
} else {  
    x=5;  
}
```

If we select branch testing, we will write test cases giving the following values as input to the function:

- i) 0; ii) 3.

Test Case Selection

There are two ways to define test cases, depending on the employed strategy.

White-box. A test case selection approach that allows defining tests **only based on the implementation** of the functionalities.

It is also called structural testing: developers exercise how the system is done and **not how it should be done**.

This strategy cannot determine if use cases are documented but not implemented.

If someone asks you the difference between black- and white-box testing, **do not answer** that *black-box testing focuses on the I/O behavior of the code while white-box testing focuses on the implementation*.

Both types of testing focus on I/O behavior. Also, in white-box testing, you test if the output is the one expected in response to certain inputs. The distinction is how **to select tests** based on the specification or the implementation.

Black-box. A test case selection approach that allows defining tests **only based on the specification** of the system.

It is also called functional testing: developers exercise what the system allows them to do, **not how it implements functionalities**.

This strategy cannot determine if there are undocumented use cases that have been implemented.

Black-box Test Case Selection: Equivalence Class Testing

Equivalence class testing is a straightforward approach to select test cases (a.k.a. equivalence partitioning).

It groups and partitions the test input data to create classes that resemble the specified requirements and common behavior or attribute(s) of the aggregated inputs. Test cases are created for each equivalence class.

There are two basic types of equivalence class testing: **weak** and **strong** partitioning.

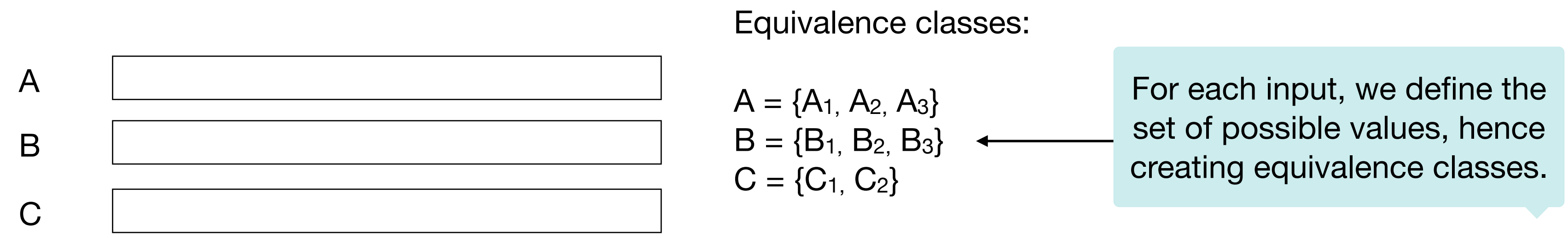
With the former, the team tests **one variable** of each equivalence class, i.e., *single fault assumption*.

With the latter, the team selects test cases for each element of the **cartesian product of the equivalence**, ensuring *completeness*, as it covers all equivalence classes and offers the team each possible combination of inputs, i.e., *multiple fault assumption*.

Equivalence class testing **reduces redundancy** in testing and makes the process agile and powerful.

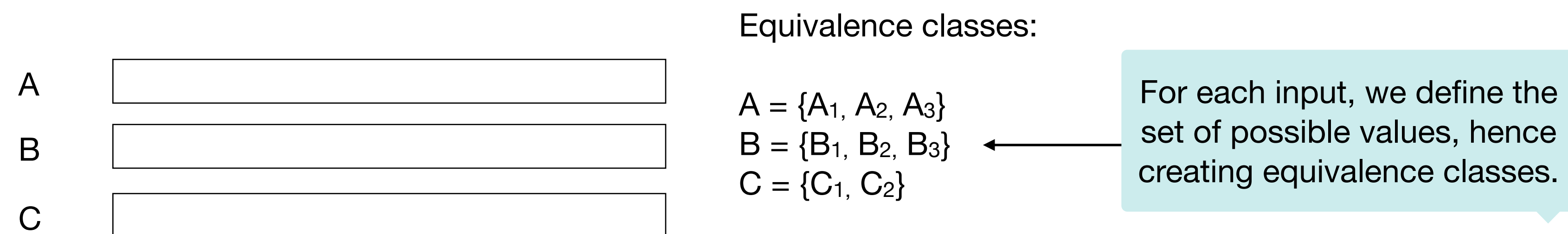
Example. Suppose we want to test functionality with three integer inputs, i.e., A, B, and C.

When applying equivalence class testing, the first step is to define classes.

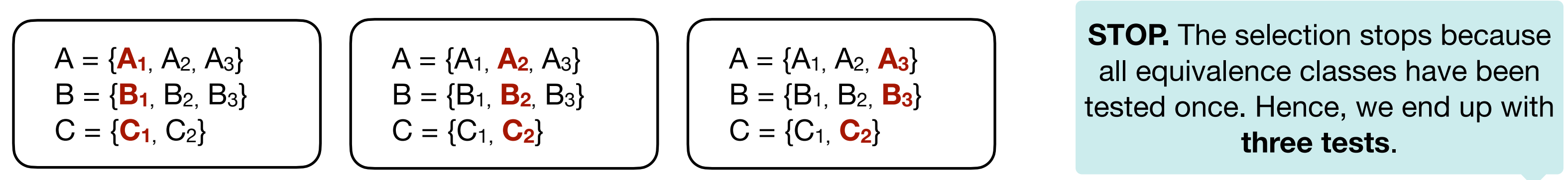


Black-box Test Case Selection: Equivalence Class Testing

Example. Suppose we want to test functionality with three integer inputs, i.e., A, B, and C.
When applying equivalence class testing, the first step is to define classes.



Weak equivalence class testing. We ensure that all partitions are covered at least once.

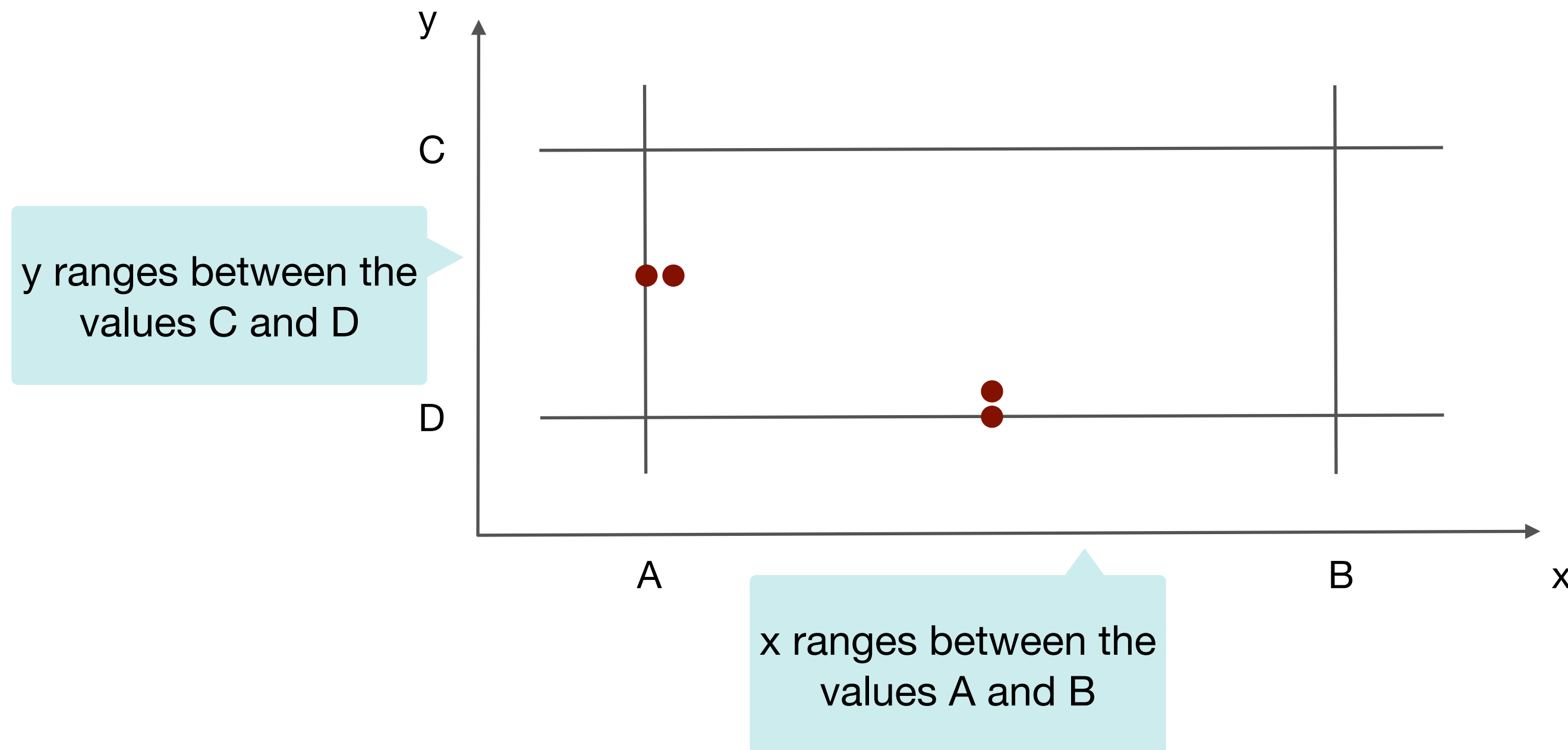


Strong equivalence class testing. We ensure the creation of several test cases equal to the cartesian product.
In the example, we should define **18 test cases** to exercise all partitions in combination with the others.

This type of testing assumes that the variables are independent of each other. However, if two variables are semantically related, with the strong equivalence class testing, we would test combinations of values that make no sense, leading to the explosion of the number of tests.

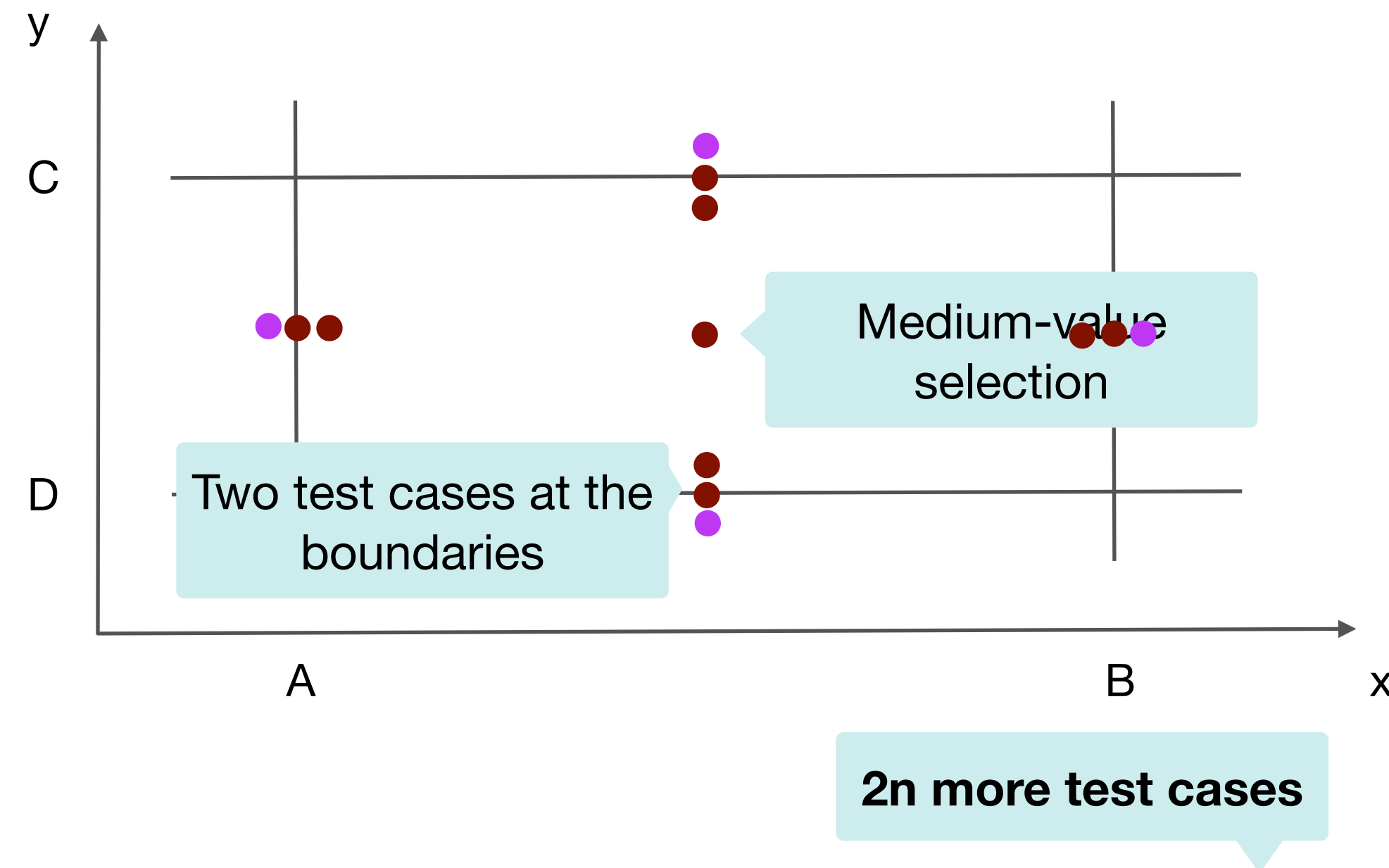
Black-box Test Case Selection: Boundary Testing

Most parts of software programs have erroneous behavior at the boundary of two equivalence classes. Hence, we would need to reinforce equivalence partitioning.



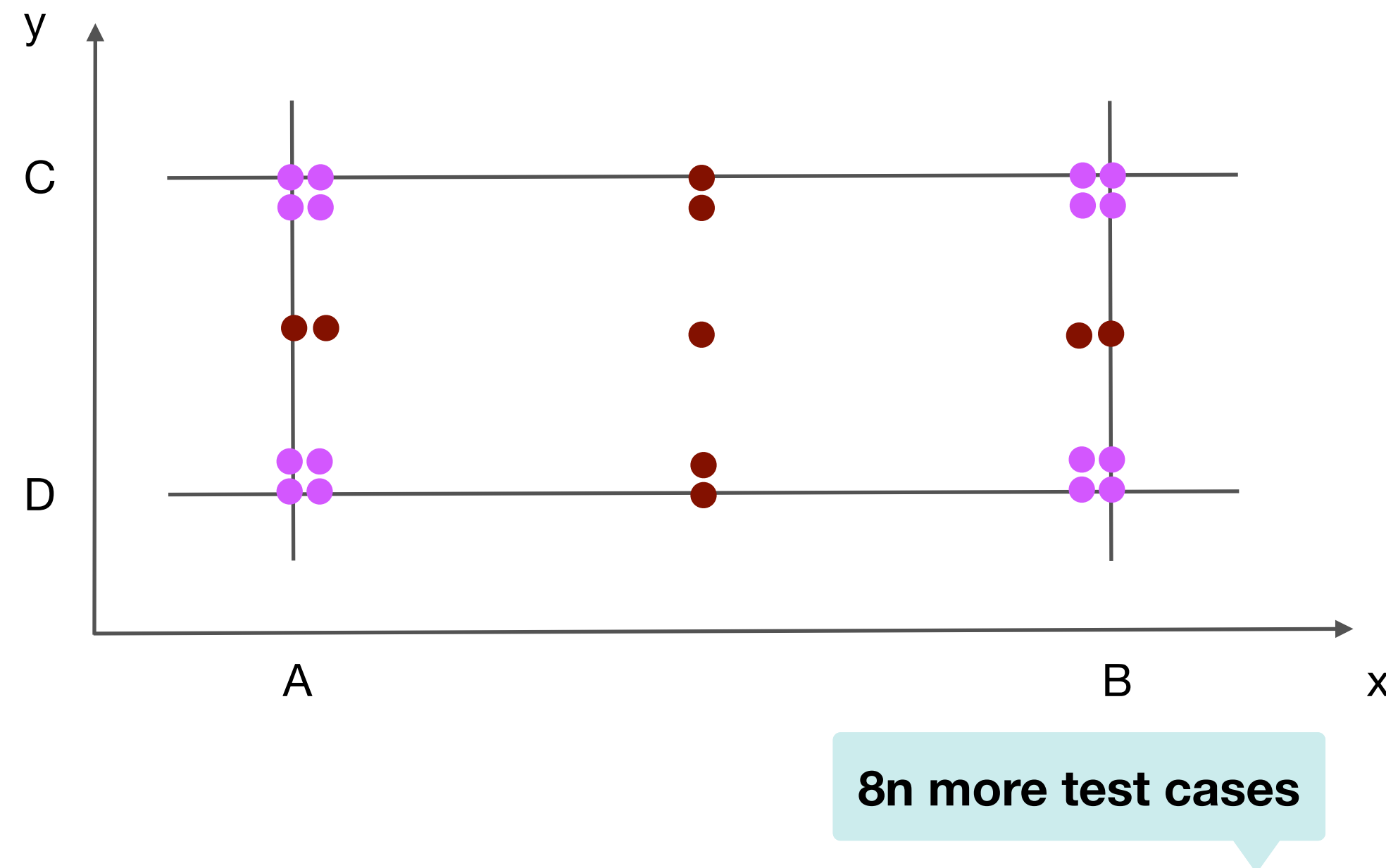
Black-box Test Case Selection: Boundary Testing

Most parts of software programs have erroneous behavior at the boundary of two equivalence classes. Hence, we would need to reinforce equivalence partitioning.



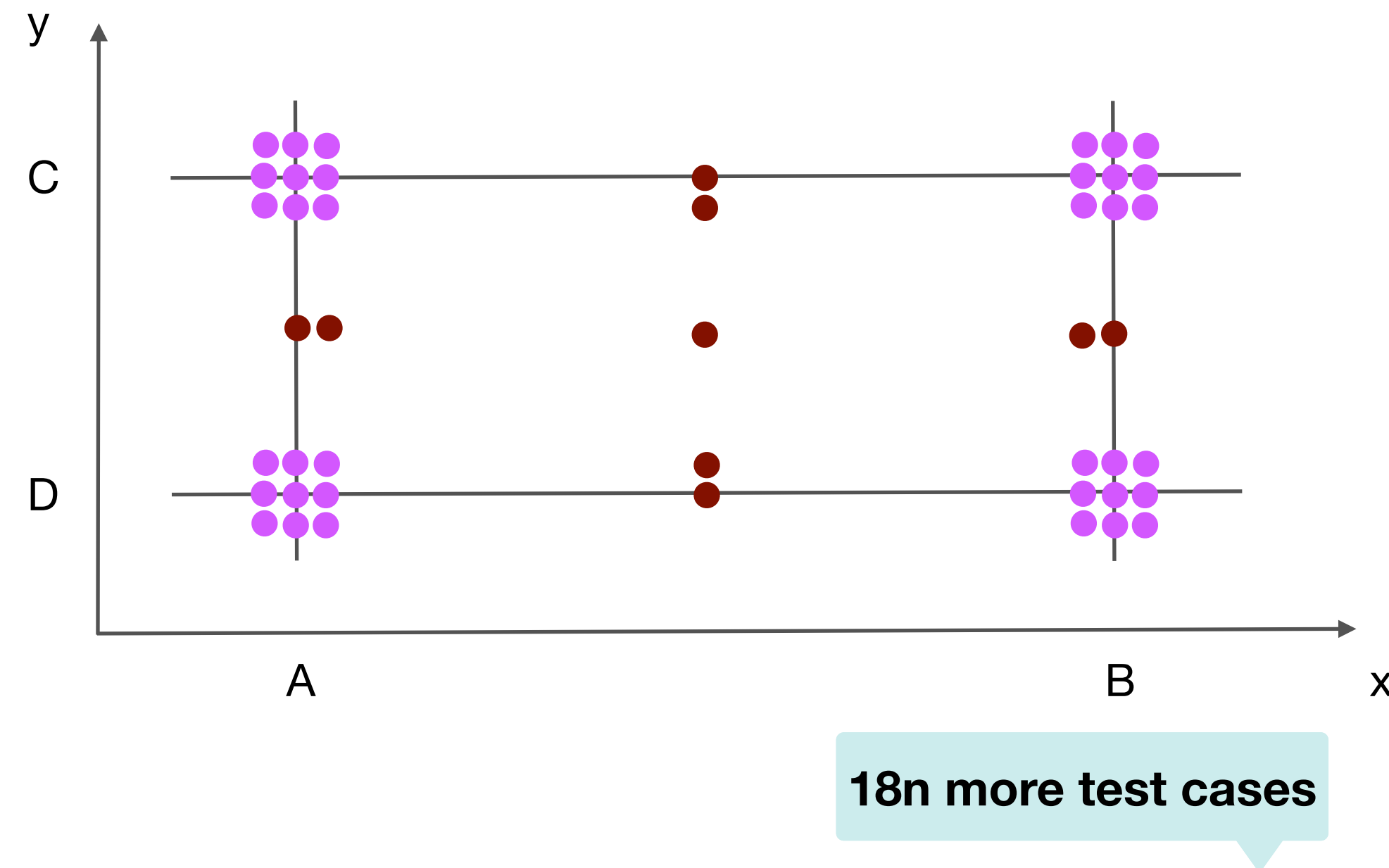
Black-box Test Case Selection: Boundary Testing

Most parts of software programs have erroneous behavior at the boundary of two equivalence classes. Hence, we would need to reinforce equivalence partitioning.



Black-box Test Case Selection: Boundary Testing

Most parts of software programs have erroneous behavior at the boundary of two equivalence classes. Hence, we would need to reinforce equivalence partitioning.



Black-box Test Case Selection: Category Partition

The techniques discussed so far can lead to defining too few or too many test cases. Furthermore, these techniques do not consider the meaning of the variables.

Category Partition.

1. The system is first divided into functionalities.
2. Each functionality is then divided into parameters.
3. Each parameter has its categories.

A

For a numeric input A, suppose it ranges between 1 and 19. This characteristic is the only one to consider.

B

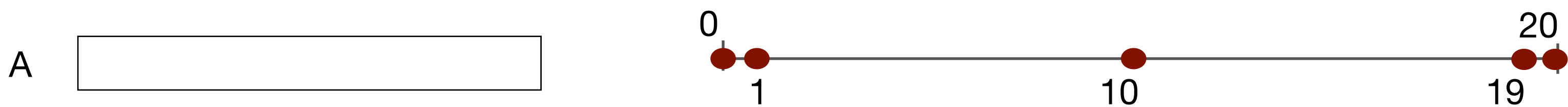
For an alphanumeric input B, suppose it represents the fiscal code with a fixed length of 15 characters. Generally, alphanumeric inputs are characterized by length, format, and - in some cases - by their values.

Black-box Test Case Selection: Category Partition

With the techniques discussed so far, we lead to the definition of too few or too many test cases.
Furthermore, these techniques do not consider the meaning of the variables.

Category Partition.

- 1. The system is first divided into functionalities.
- 2. Each functionality is then divided into parameters.
- 3. Each parameter has its categories.



- A:
- (1) A = 0; [error].
 - (2) A = 1; [property valueOk].
 - (3) A = 10; [property valueOk].
 - (4) A = 19; [property valueOk].
 - (5) A = 20; [error].

We introduce semantic relations, i.e., the format is conceptually combined with the length of the field.

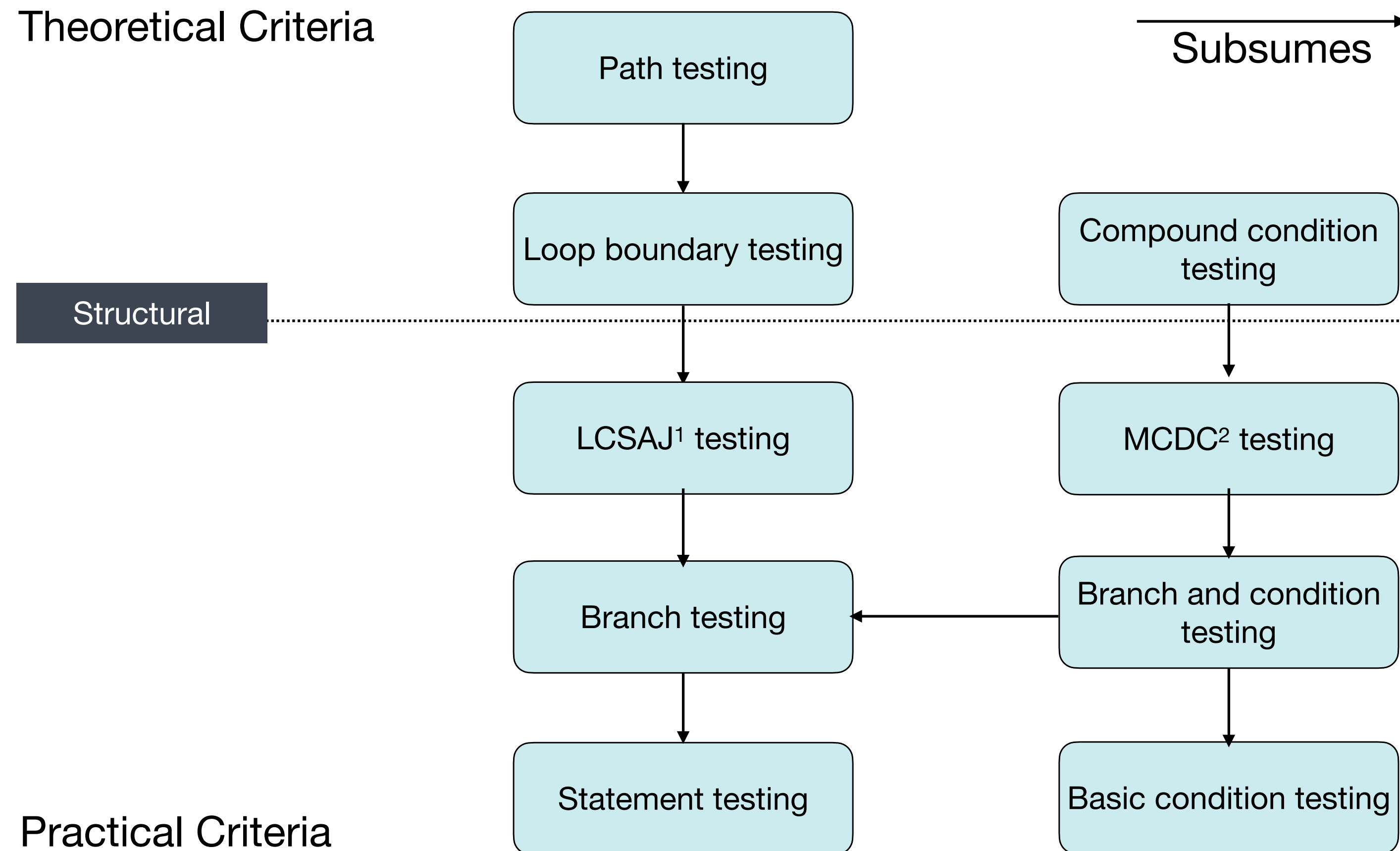
B

- B:
- (1) Length < 15; [error].
 - (2) Length = 15; [property lengthOk].
 - (3) Length > 15; [error].
 - (4) Format = [^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]\$]; [if lengthOk, property formatOk].
 - (5) Format != [^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]\$]; [error].

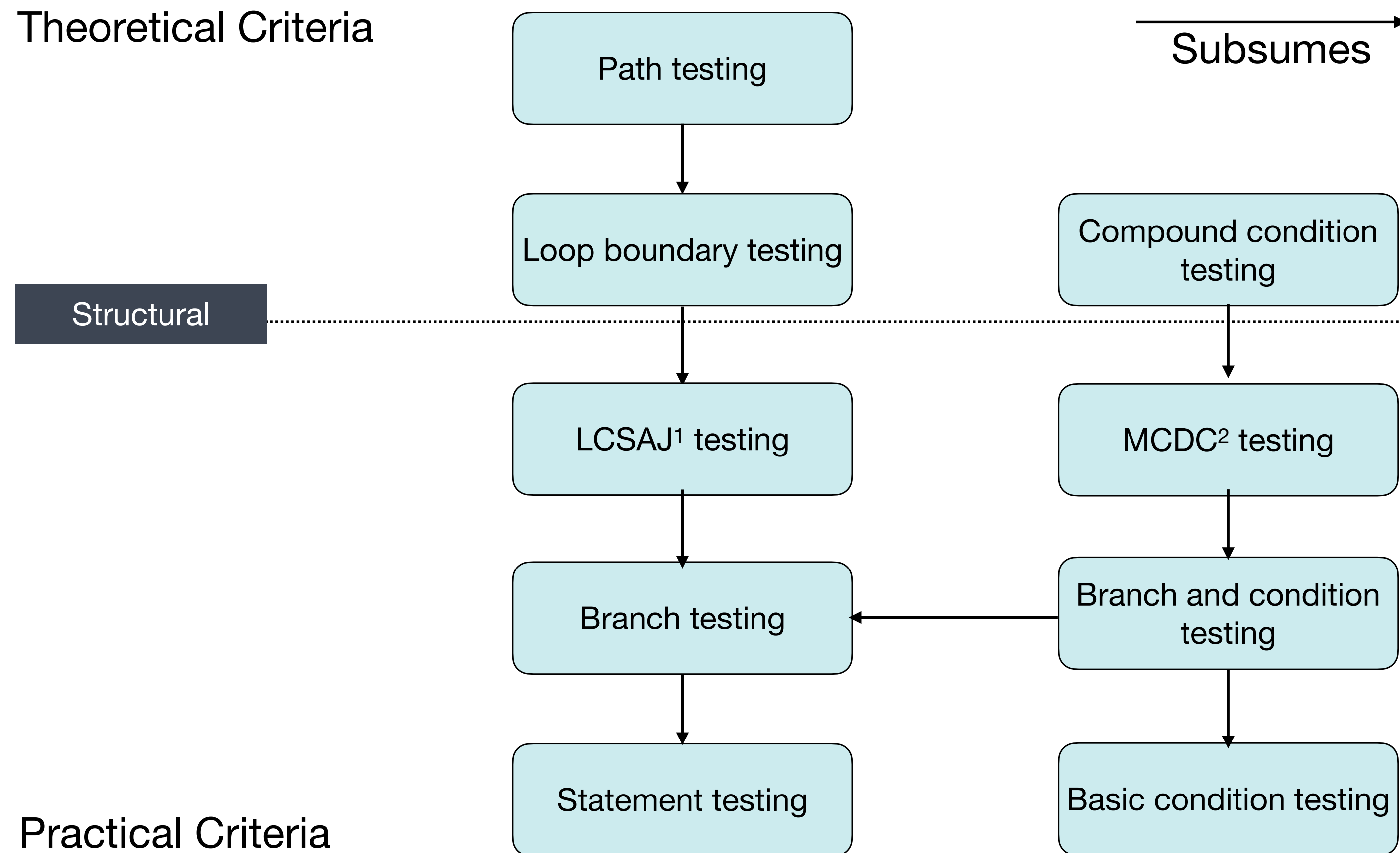
More inputs are combined, e.g., more (alpha)numeric inputs.

White-box Test Case Selection

Theoretically, the techniques used for white-box test case selection are easier than those used in black-box testing. They are related to a test code coverage criterion.



White-box Test Case Selection



Predicate

```
if ( (a > b) || ( c ) && p(x) )  
    o.M();  
else  
    o.N();
```

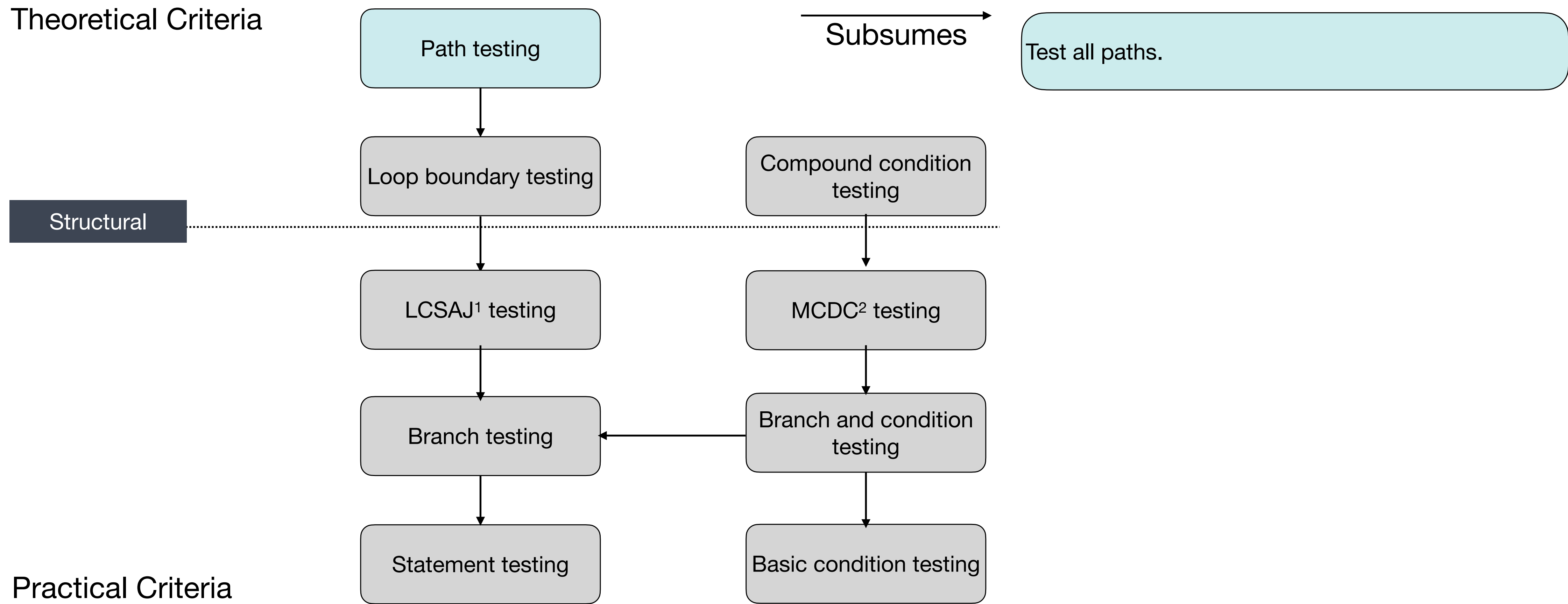
Clause

```
if ( (a > b) || ( c ) && p(x) )  
    o.M();  
else  
    o.N();
```

¹Linear code sequence and jump

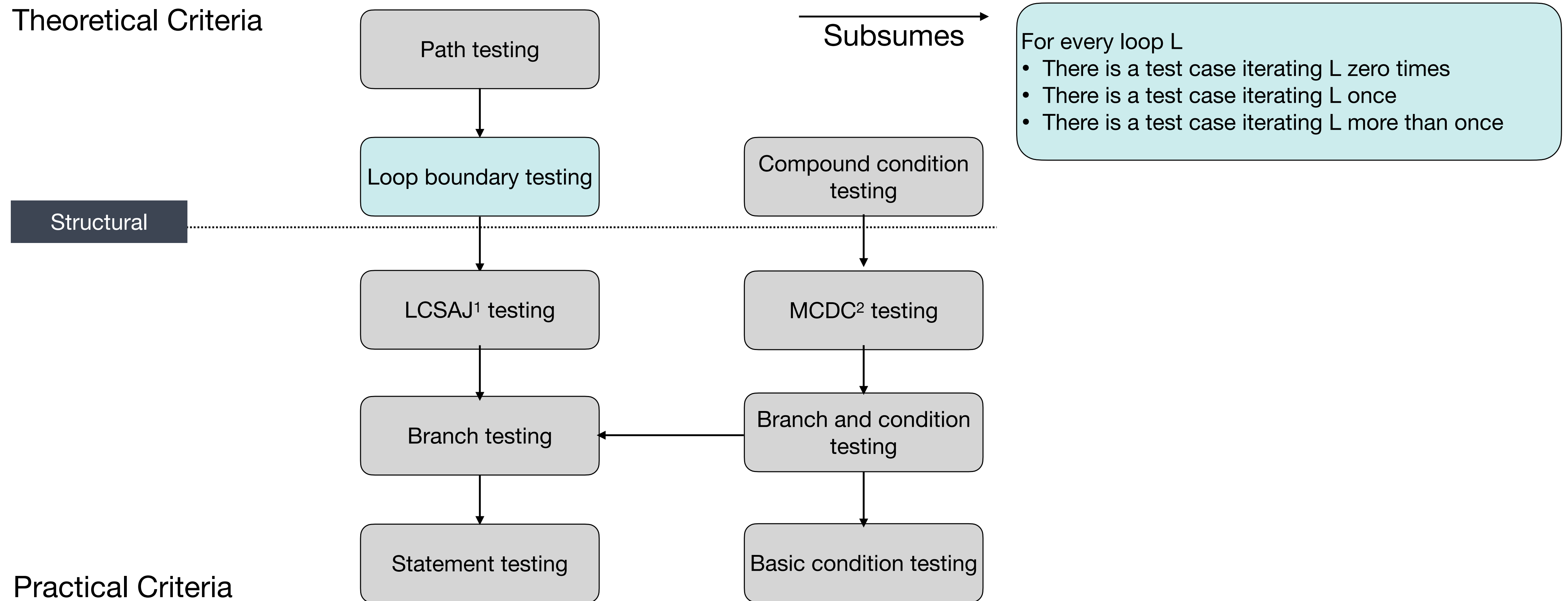
2Modified condition/decision coverage

White-box Test Case Selection



¹Linear code sequence and jump
²Modified condition/decision coverage

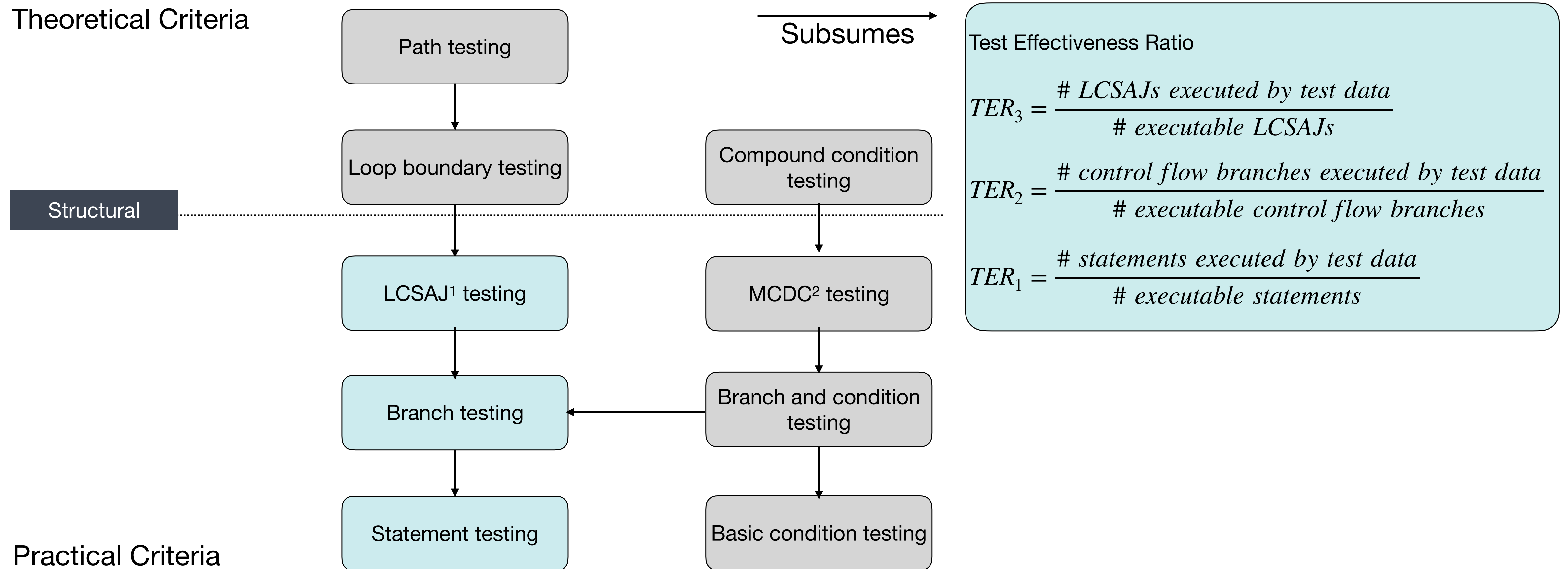
White-box Test Case Selection



¹Linear code sequence and jump

2Modified condition/decision coverage

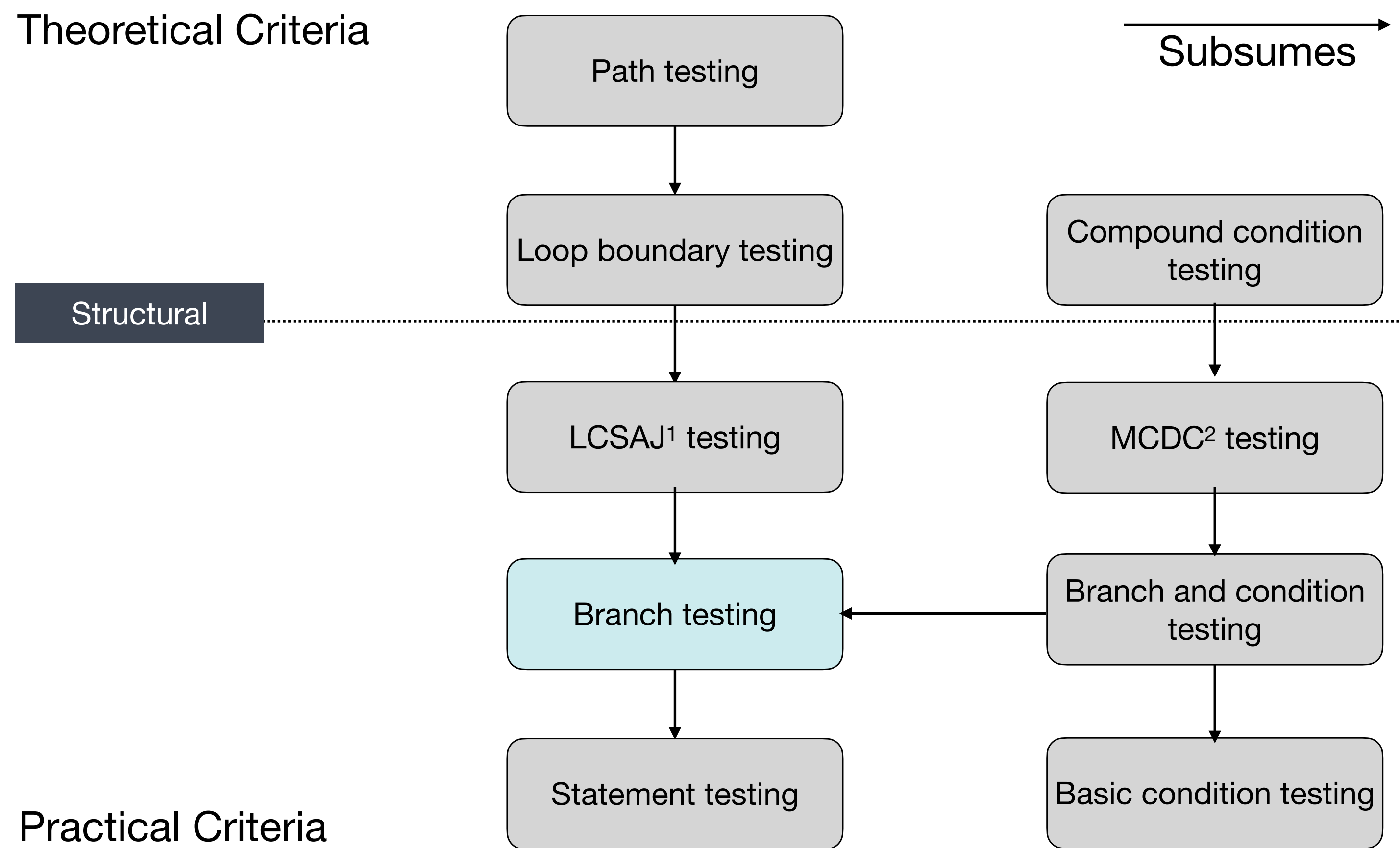
White-box Test Case Selection



¹Linear code sequence and jump

2Modified condition/decision coverage

White-box Test Case Selection



$$TER_2 = \frac{\# \text{ control flow branches executed by test data}}{\# \text{ executable control flow branches}}$$

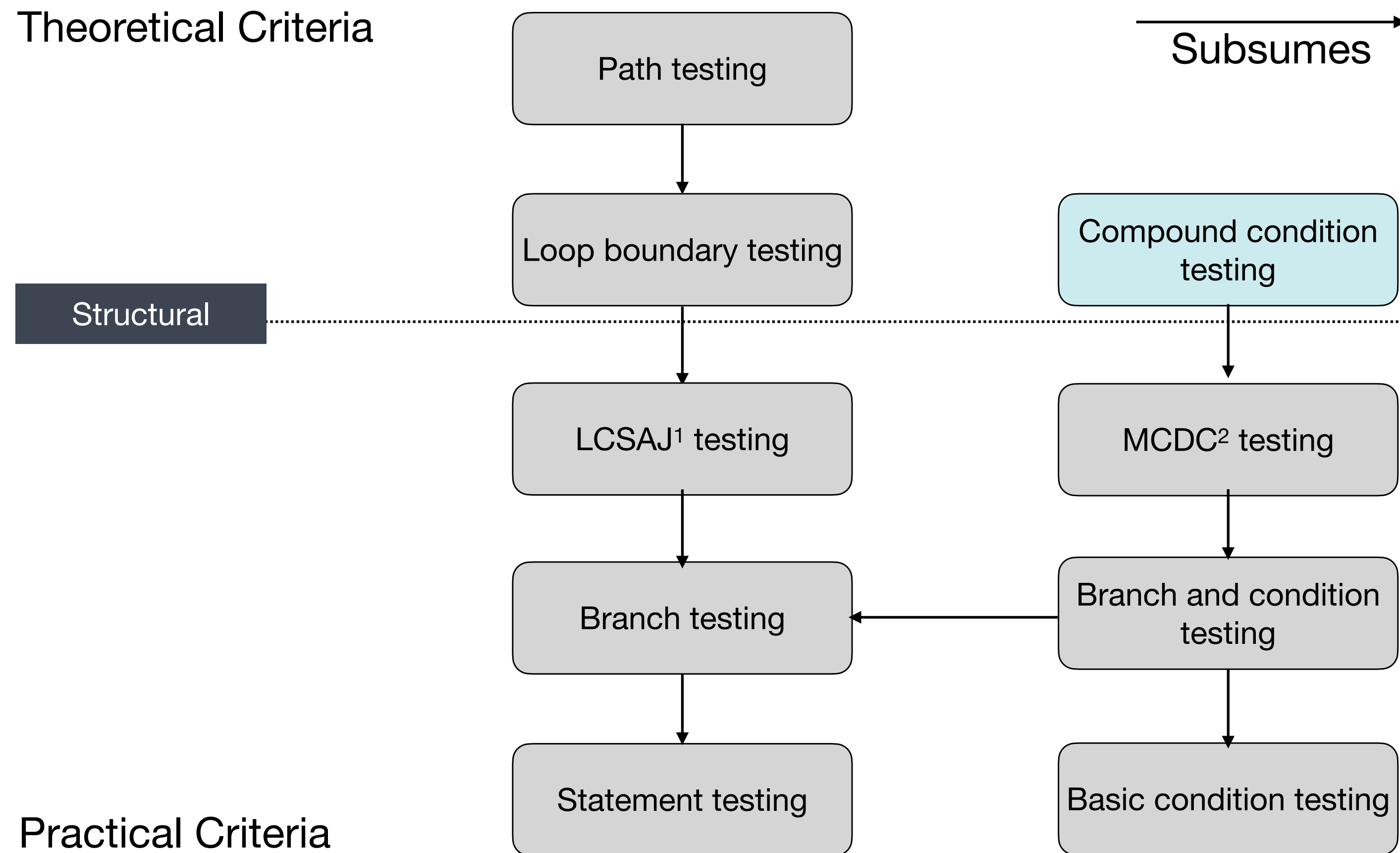
We consider at least (i) one test where the predicate is evaluated as true and (ii) one test where the predicate is evaluated as false.

$$A \wedge (B \vee C)$$

	A	B	C	Res.
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

¹Linear code sequence and jump
²Modified condition/decision coverage

White-box Test Case Selection



Test all condition combinations.

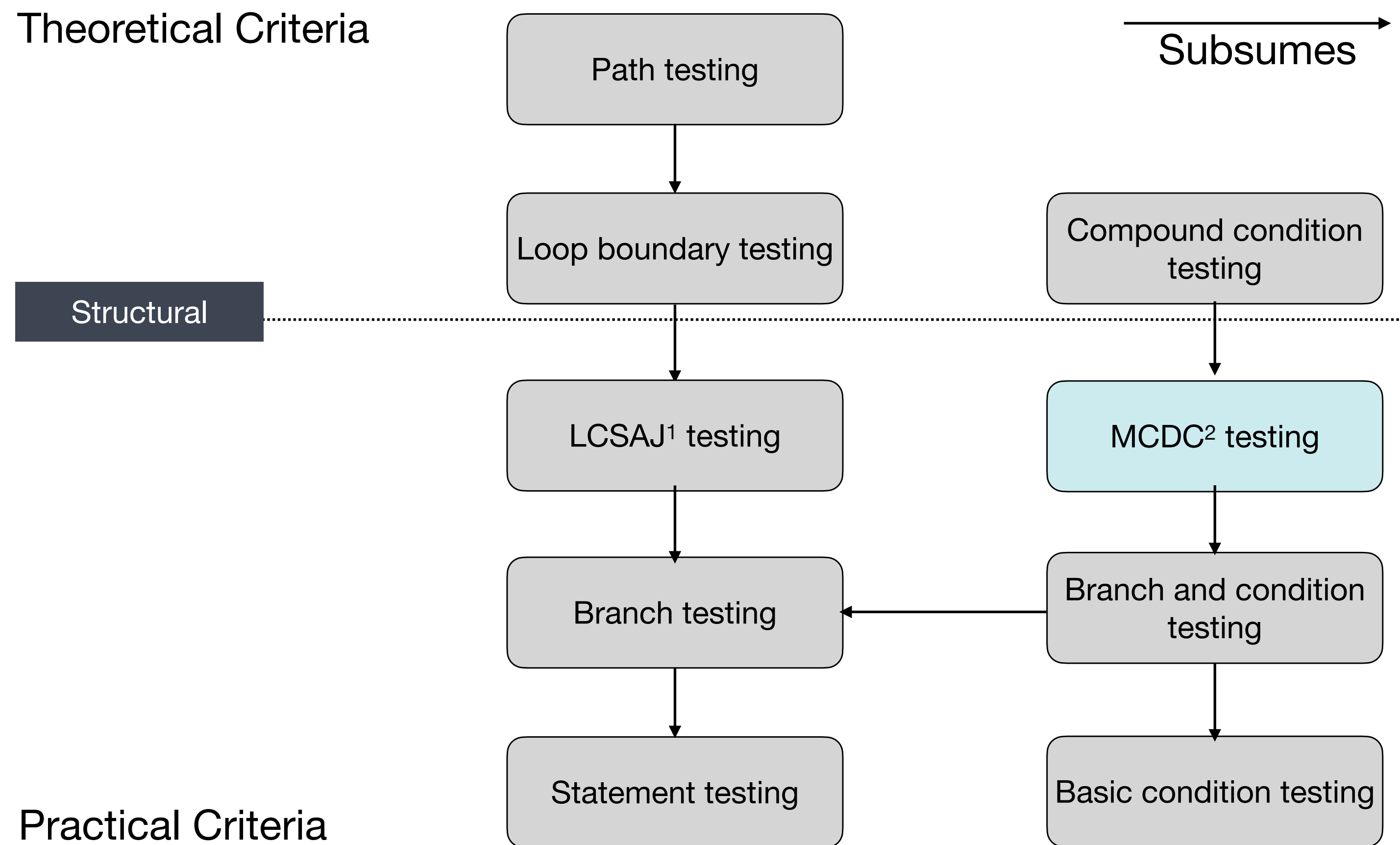
$$A \wedge (B \vee C)$$

	A	B	C	Res.
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

¹Linear code sequence and jump

2Modified condition/decision coverage

White-box Test Case Selection



We consider for each predicate only combinations of clauses where one clause determines the truth value of the decision.

$$A \wedge (B \vee C)$$

	A	B	C	Res.
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

$A : \{1,5\}, \{2,6\}, \{3,7\}$

$B : \{2,4\}$

$C : \{3,4\}$

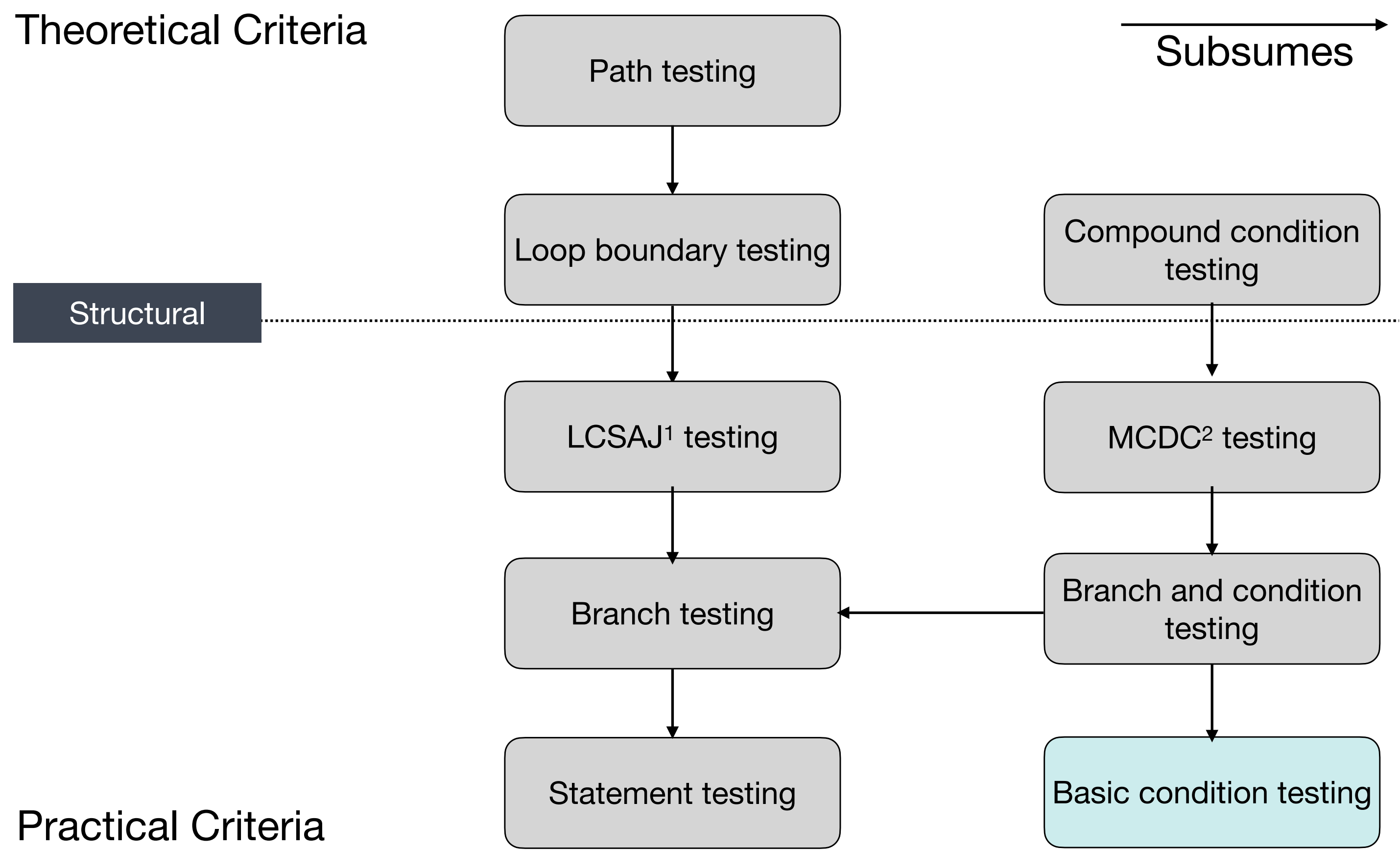
Minimum set $\{2,3,4,6\}$ or $\{2,3,4,7\}$

The number of test cases is $N+1$ for N clauses.

¹Linear code sequence and jump

²Modified condition/decision coverage

White-box Test Case Selection



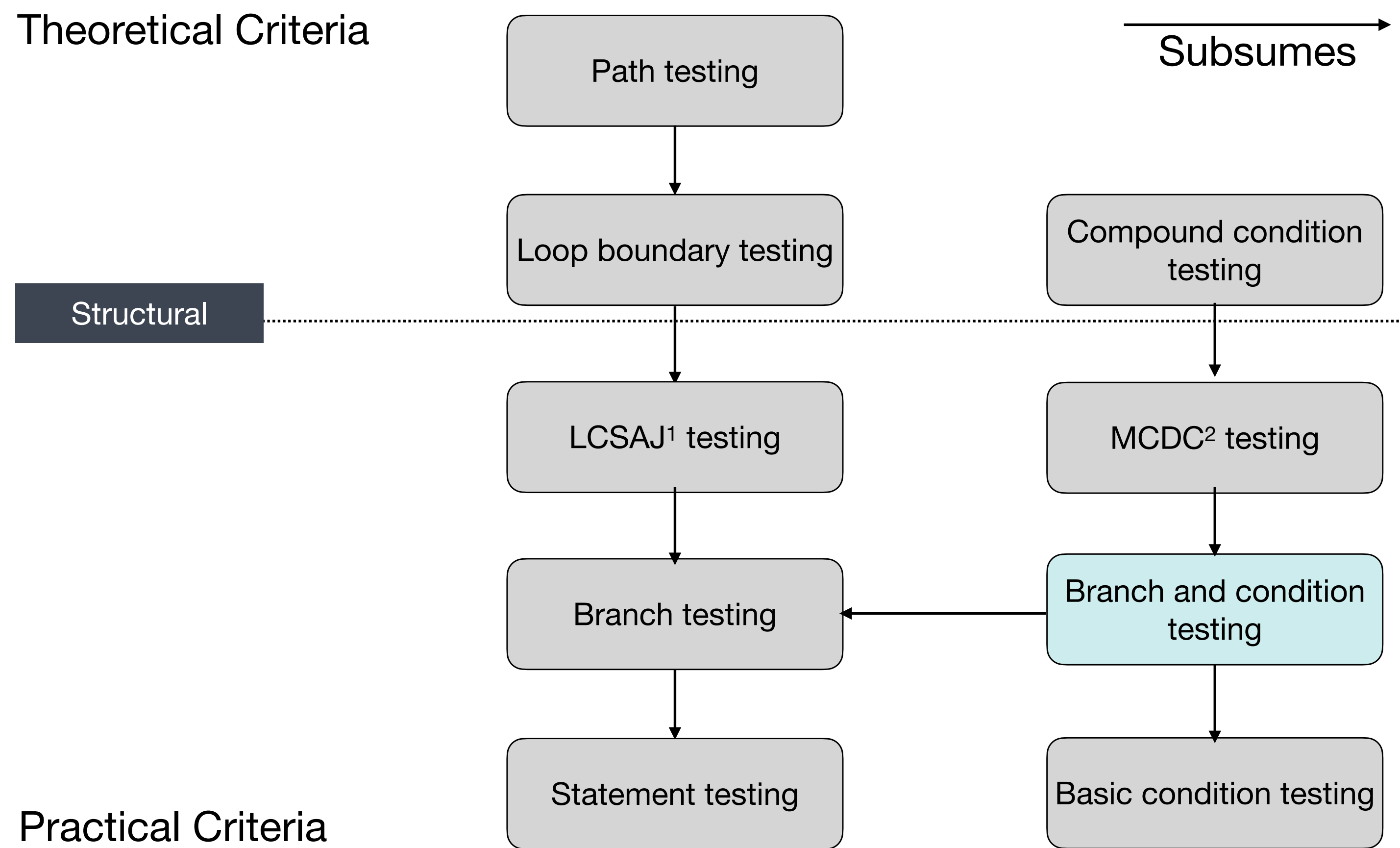
We consider at least (i) one test where each clause is evaluated as true and (ii) one test where each clause is evaluated as false.

$$A \wedge (B \vee C)$$

	A	B	C	Res.
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

¹Linear code sequence and jump
²Modified condition/decision coverage

White-box Test Case Selection



Test all branches and conditions.

We consider for each predicate at least (i) one test where it is evaluated true and (ii) one test where it is evaluated false.

Furthermore, we consider for each clause in a predicate, at least (i) one test where the clause is evaluated as true and (ii) one test where the clause is evaluated as false.

$$A \wedge (B \vee C)$$

	A	B	C	Res.
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

¹Linear code sequence and jump
²Modified condition/decision coverage

Integration Test Case Selection

Once completed the selection of unit tests, we can exercise the integration of multiple components. Integration testing aims at verifying if **multiple components work together correctly**.

Two key concepts of integration testing are **test stubs and drivers**.

Test stubs are pieces of code that partially implement a component on which **the component under test depends**.

Drivers are partial implementations of a component **that depend on the component under test**.

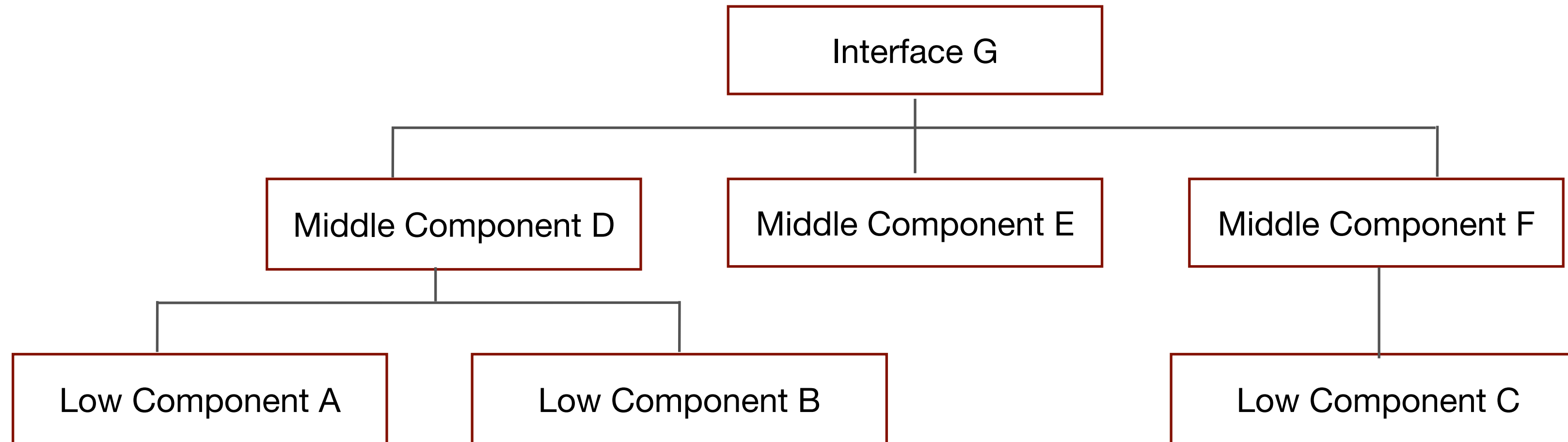
There are various techniques to proceed with integration testing, e.g., **Big Bang, Bottom-Up, Top-Down, Sandwich, and Modified Sandwich**.

The technique selection may depend on specific aspects of a software project.

For instance, it has been shown that **most integration errors happen when exercising the GUI**. As such, if one has a system strongly dependent on the GUI, they might want to prefer testing GUI interactions first.

Integration Test Case Selection

Big Bang Integration Testing



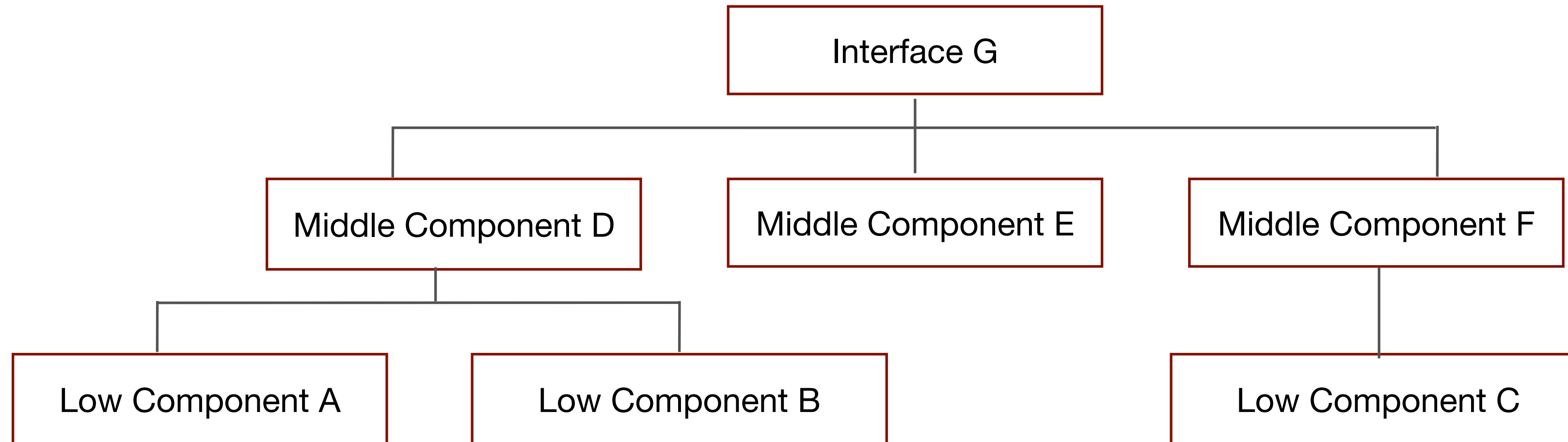
All components are put together at the same time.

While quick, it is not recommended: localizing the issues is challenging if an integration fault arises.

It is better to reason on how to integrate components step-by-step.

Integration Test Case Selection

Bottom-Up Integration Testing



Subsystems of **lower layers** are integrated first. The integration then proceeds with the **higher layers**. **Drivers** could be needed to test the integration. In the example above, there are four steps:

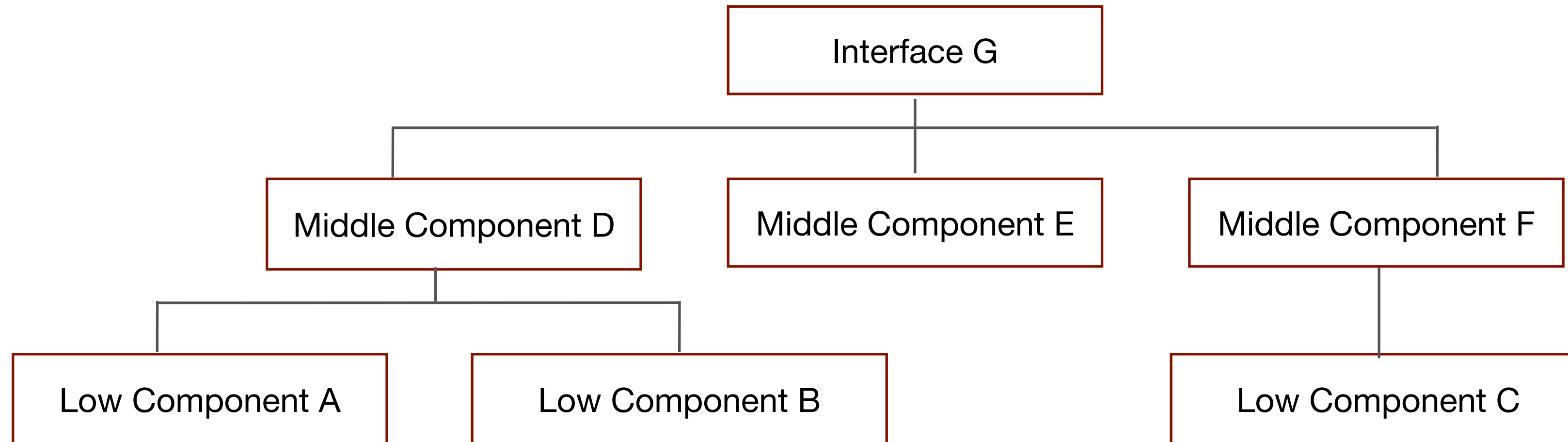
1. Unit testing of A, B, and C;
2. Integration testing of A, B, and D;
3. Integration testing of C and F;
4. Integration testing of all components.

The interface is tested as the last step.

If a system has performance constraints, the more delicate subsystems are integrated first.

Integration Test Case Selection

Top-Down Integration Testing



Subsystems of **higher-layers** are integrated first. The integration then proceeds with the **bottom-layers**. **Stubs** could be needed to test the integration. In the example above, there are three steps:

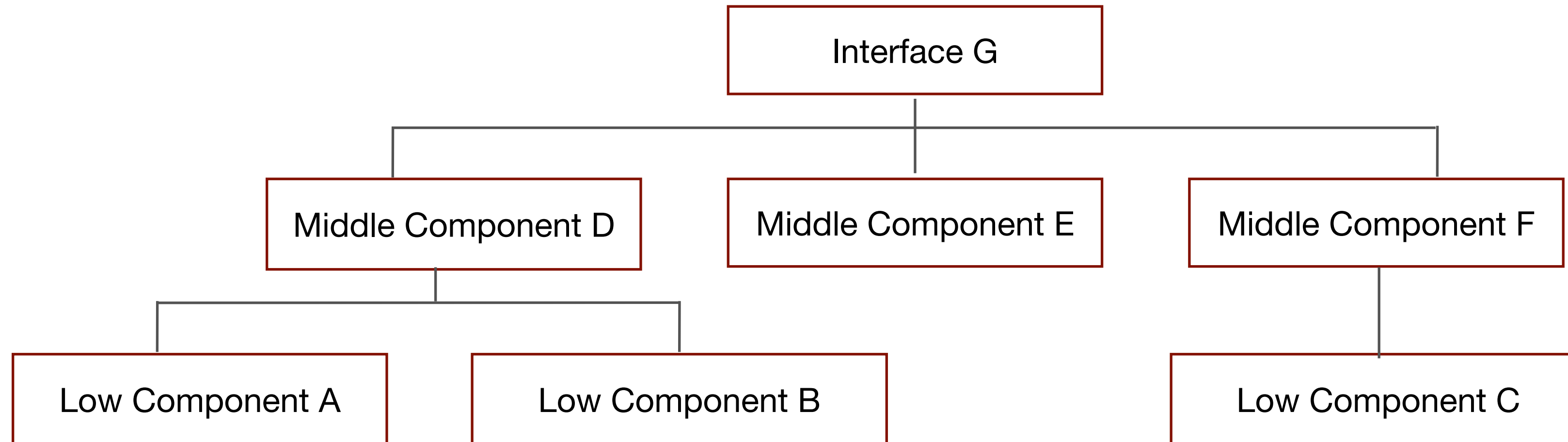
1. Unit testing of G;
2. Integration testing of G, D, E, F;
3. Integration testing of all components.

Writing stubs could be hard.

The technique is good to test the GUI asap.

Integration Test Case Selection

Sandwich Integration Testing



Subsystems in the **middle** are integrated first. The integration then proceeds, aiming at minimizing the number of **drivers/stubs** to develop. In the example above, there are four steps:

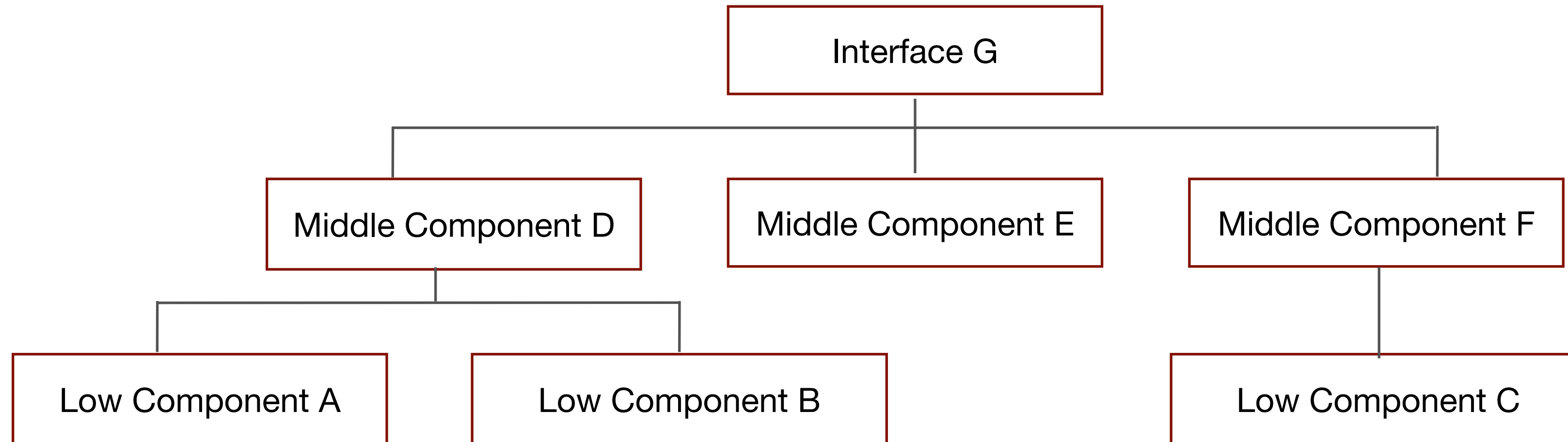
1. Unit testing of A, B, C, and G;
2. Integration testing of A, B, and D;
3. Integration testing of F and C;
4. Integration testing of all components.

Bottom-Up and Top-Down approaches are conducted in parallel.

Components in the middle are not individually tested.

Integration Test Case Selection

Modified Sandwich Integration Testing



Inspired by the original Sandwich method, it includes the **unit testing of middle components**. In the example above, there are four steps:

1. Unit testing of all components;
2. Integration testing of A, B, and D;
3. Integration testing of F and C;
4. Integration testing of all components.

Bottom-Up and Top-Down approaches are conducted in parallel.

All components are first individually tested.



Break